CA4 - Telecom Customer Churn Classification

Group 3 Members

- Stig Espen Paulsberg
- Vinicius Millan
- Zaur Kambiev

Kaggle team `Group 3`

1. Imports

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.feature_selection  import SequentialFeatureSelector
from sklearn.metrics            import confusion_matrix, make_scorer, pre
from sklearn.metrics.pairwise   import laplacian_kernel
from sklearn.model_selection    import GridSearchCV, StratifiedKFold, tra
from sklearn.pipeline           import Pipeline
from sklearn.preprocessing      import StandardScaler
from sklearn.svm                import SVC
from xgboost                    import XGBClassifier
from sklearn.compose            import ColumnTransformer
from sklearn.decomposition      import PCA
```

2. Reading data

In [2]:
```
df = pd.read_csv("data/trainCA4DAT200.csv", index_col="CustomerId")
df.head()
```

Out[2]:

CustomerId	State	AccountLength	AreaCode	InternationalPlan	VoiceMailPlan	Nu
952920	NJ	87	510	no	no	
895150	RI	114	510	no	yes	
636987	ND	55	415	no	no	
538188	DC	100	510	yes	no	
947531	AL	74	408	no	yes	

3. Data exploration & visualisation

- Check missing and duplicated values.

In [3]:
```
print(f"Missing values: {df.isna().sum().sum()}")
print(f"Duplicated rows: {df.duplicated().sum()}")
```

```
Missing values: 0
Duplicated rows: 0
```

3.a Categorical variables - overview

- Check the number of unique values in the categorical features.

In [4]:
```
raw_categorical = ["State", "AreaCode", "InternationalPlan", "VoiceMailPl

print(
    df[raw_categorical]
    .nunique()
    .to_string()
)
```

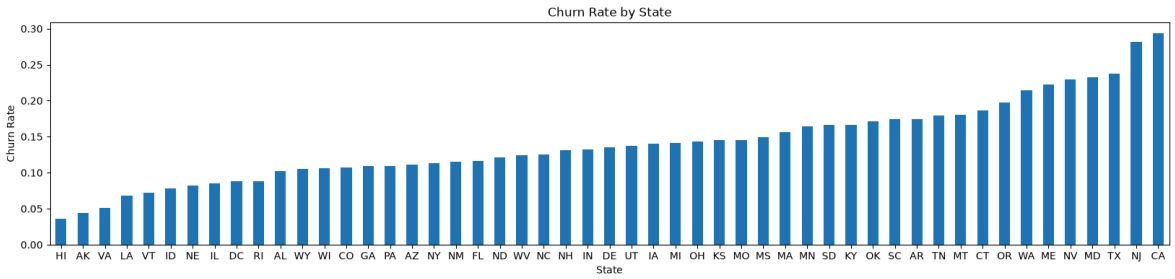
```
State                51
AreaCode              3
InternationalPlan     2
VoiceMailPlan         2
```

- How is `State` connected with the churn rate?

In [5]:
```
plt.figure(figsize=(20, 4))

(
    df.groupby("State")
    ["Churn"].mean()
    .sort_values()
    .plot.bar()
)

plt.xlabel("State")
plt.tick_params(axis="x", rotation=0)
plt.ylabel("Churn Rate")
plt.title("Churn Rate by State")
plt.show()
```



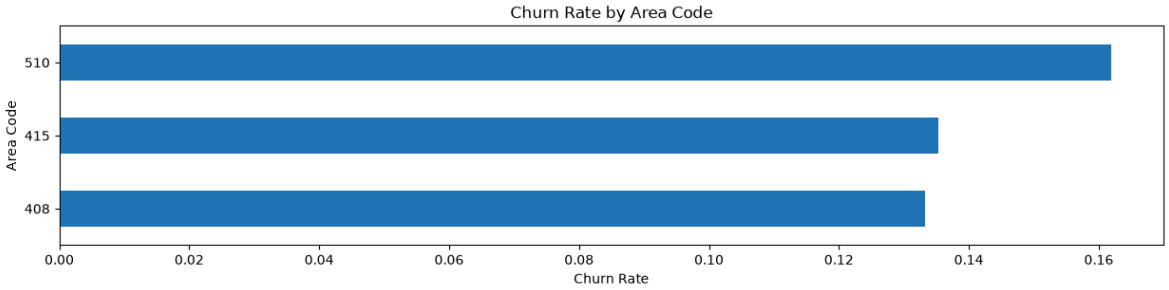
- It seems that `State` is a good predictor of `Churn`, with the churn rate varying from 4% to 30% across states. However, this could also simply reflect the characteristics of the people living in those states.

- How is `AreaCode` connected with the churn rate?

In [6]:
```
plt.figure(figsize=(15, 3))

(
    df.groupby("AreaCode")
    ["Churn"].mean()
    .plot.barh()
)

plt.xlabel("Churn Rate")
plt.ylabel("Area Code")
plt.title("Churn Rate by Area Code")
plt.show()
```



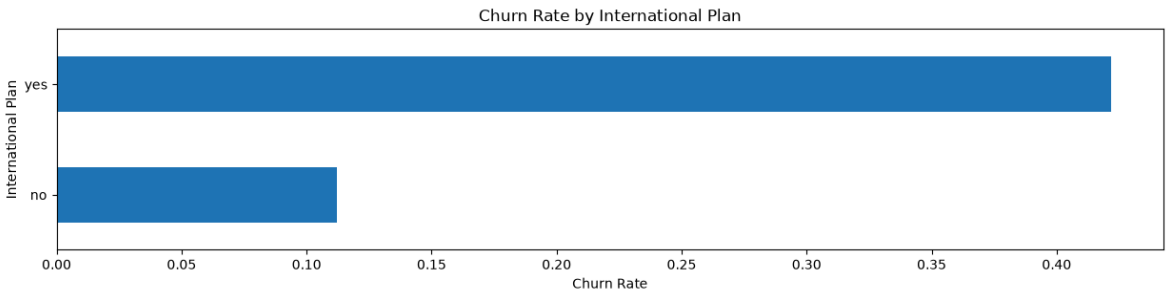
- `AreaCode` has no noticeable impact on the churn rate.

- How is `InternationalPlan` connected with the churn rate?

In [7]:
```
plt.figure(figsize=(15, 3))

(
    df.groupby("InternationalPlan")
    ["Churn"].mean()
    .plot.barh()
)

plt.xlabel("Churn Rate")
plt.ylabel("International Plan")
plt.title("Churn Rate by International Plan")
plt.show()
```

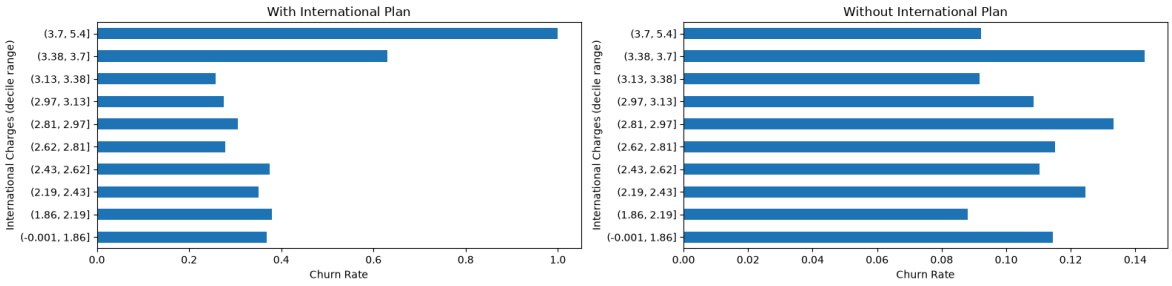


- Here we see a large difference in churn rates. Let's compare customers with different levels of international charges.

In [8]:
```
fig, axes = plt.subplots(1, 2, figsize=(16, 4))

for value, ax in zip(["yes", "no"], axes):
    (
        df[df["InternationalPlan"] == value]
        .groupby(pd.qcut(df["IntlCharge"], q=10))
        ["Churn"].mean()
        .plot.barh(ax=ax)
    )
    ax.set_xlabel("Churn Rate")
    ax.set_ylabel("International Charges (decile range)")
    ax.set_title(
        "With International Plan" if value == "yes" else "Without Interna
    )

plt.tight_layout()
plt.show()
```



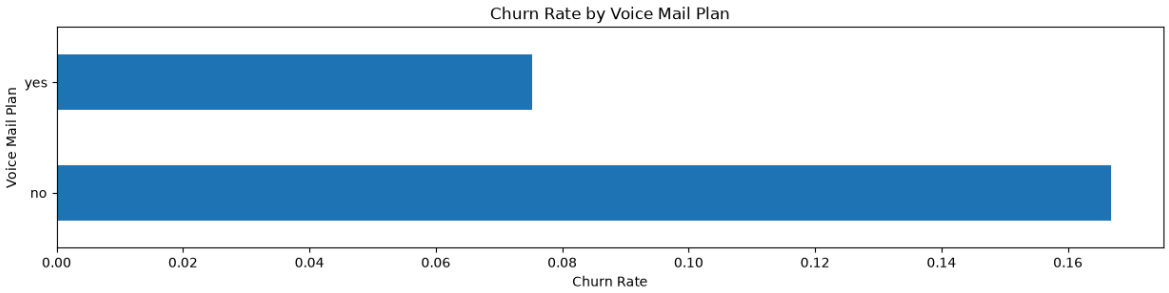
- Among customers without an international plan, churn stays roughly the same no matter how high their international charges are.
- Among customers with an international plan, churn rises with charges: every customer in the top 10% of international charges churned.

- How is `VoiceMailPlan` connected with the churn rate?

In [9]:
```
plt.figure(figsize=(15, 3))

(
    df.groupby("VoiceMailPlan")
    ["Churn"].mean()
    .plot.barh()
)

plt.xlabel("Churn Rate")
plt.ylabel("Voice Mail Plan")
plt.title("Churn Rate by Voice Mail Plan")
plt.show()
```



- Customers with a voicemail plan churn noticeably less often.

- Let's encode the binary features `InternationalPlan` and `VoiceMailPlan` as 0 and 1 so that we can include them in the correlation heatmap.

In [10]:
```
def encode_binary_features(df):
    new_df = df.copy()
    new_df["InternationalPlan"] = (df.InternationalPlan == 'yes').astype(
    new_df["VoiceMailPlan"] = (df.VoiceMailPlan == 'yes').astype(float)
    return new_df

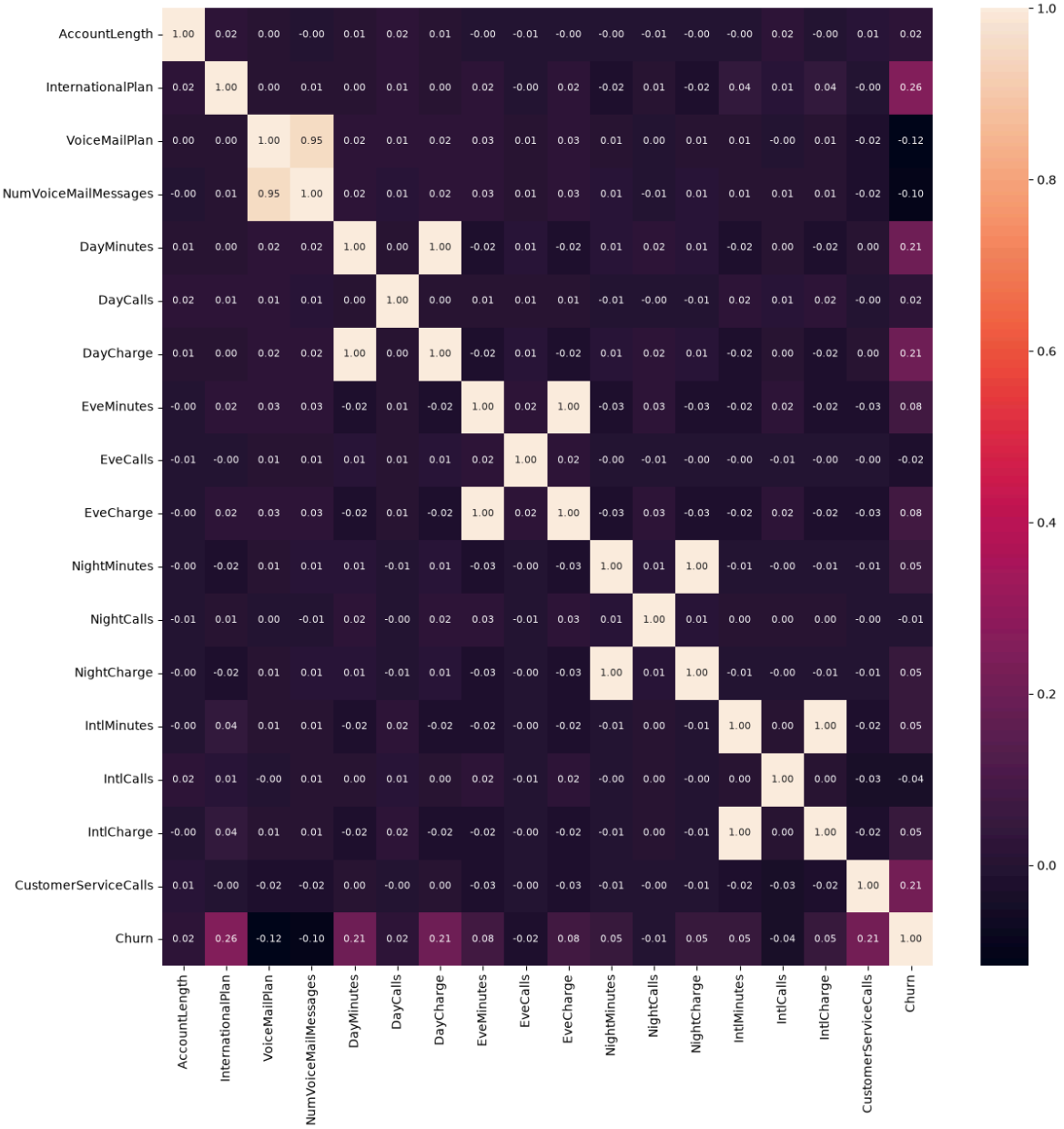
df = encode_binary_features(df)
```

3.b Numerical variables - overview

- Let's build a correlation heatmap.

In [11]:
```
heatmap_columns = df.columns.drop(["State", "AreaCode"])

plt.figure(figsize=(14, 14))
sns.heatmap(
    df[heatmap_columns].corr(),
    annot=True,
    fmt='.2f',
    annot_kws={'size': 8}
)
plt.show()
```



- Here we see that `DayMinutes`, `EveMinutes`, `NightMinutes`, and `IntlMinutes` are perfectly correlated with the corresponding charges (correlation = 1). This suggests that each charge is simply the number of minutes multiplied by a fixed per-minute rate. Since these pairs carry the same information, we can drop the charge columns to avoid redundant features.

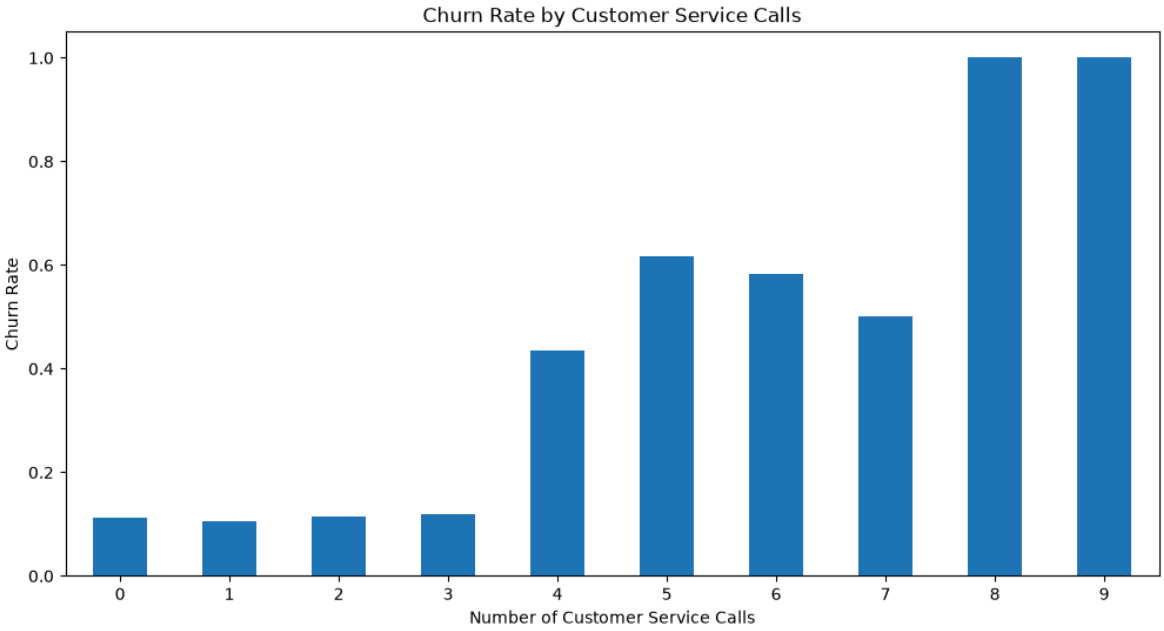
- We also see that `CustomerServiceCalls` appears to be related to `Churn`. Let's look at how the churn rate changes with the number of service calls.

In [12]:
```
plt.figure(figsize=(12, 6))

(
    df.groupby("CustomerServiceCalls")
    ["Churn"].mean()
    .plot.bar()
)

plt.xlabel("Number of Customer Service Calls")
```

```
plt.tick_params(axis="x", rotation=0)
plt.ylabel("Churn Rate")
plt.title("Churn Rate by Customer Service Calls")
plt.show()
```



- There are two large jumps in churn: from 3 to 4 service calls, the churn rate rises from about 10% to 40%, and from 7 to 8 calls, it rises from about 50% to 100%.

- Let's also check whether other numerical features contain outliers by making boxplots.

In [13]:
```
features_to_plot = [
    'AccountLength', 'NumVoiceMailMessages', 'DayMinutes',
    'DayCalls', 'EveMinutes', 'EveCalls', 'NightMinutes',
    'NightCalls', 'IntlMinutes', 'IntlCalls'
]

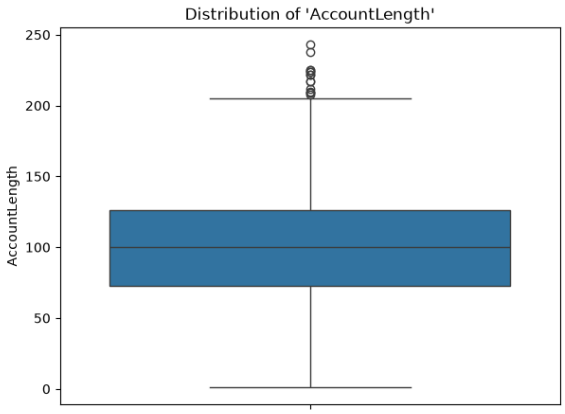
fig, axes = plt.subplots(5, 2, figsize=(15, 30))

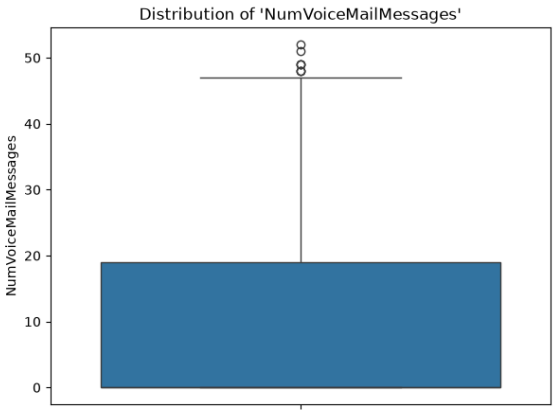
for feature, ax in zip(features_to_plot, axes.flatten()):

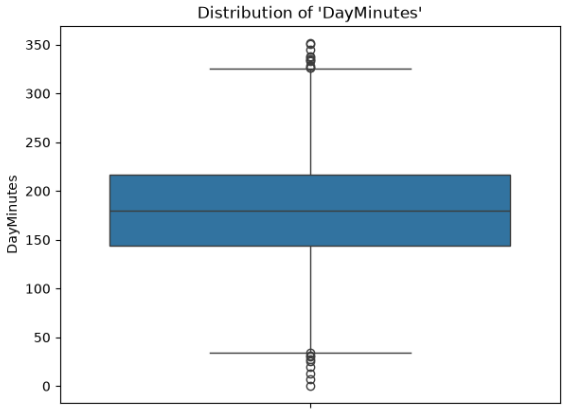
    sns.boxplot(data=df[feature], ax=ax)

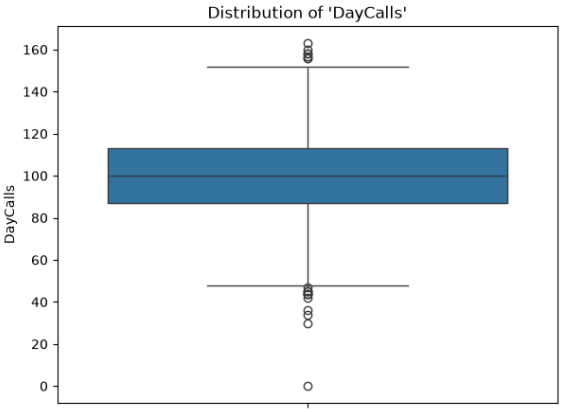
    ax.set_title(f"Distribution of '{feature}'")

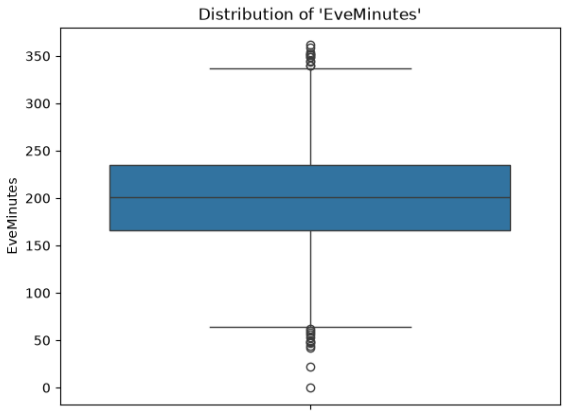
plt.show()
```

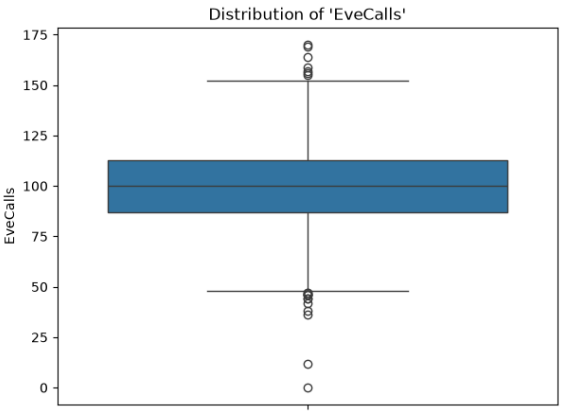

Distribution of 'AccountLength'
AccountLength
250
200
150
100
50
0

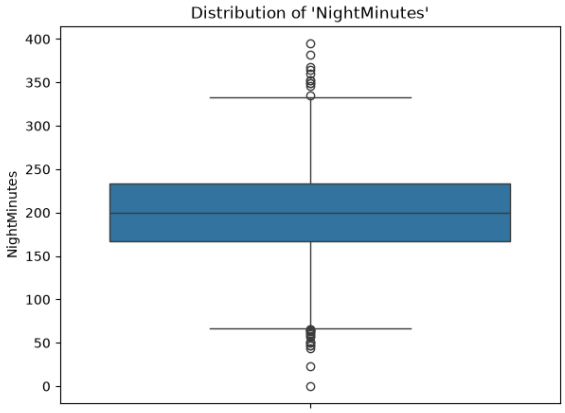

Distribution of 'NumVoiceMailMessages'
NumVoiceMailMessages
50
40
30
20
10
0

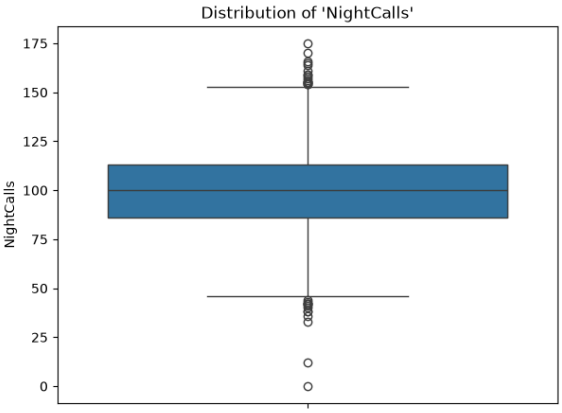

Distribution of 'DayMinutes'
DayMinutes
350
300
250
200
150
100
50
0

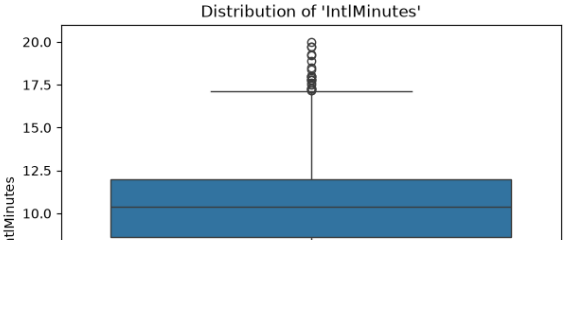

Distribution of 'DayCalls'
DayCalls
160
140
120
100
80
60
40
20
0

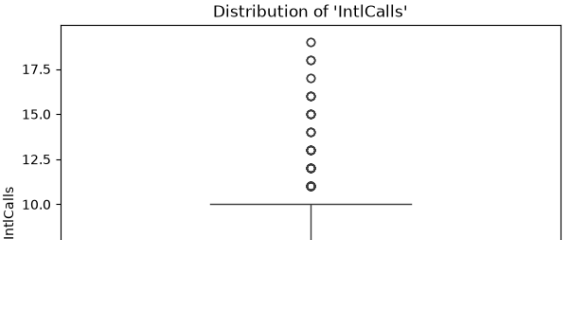

Distribution of 'EveMinutes'
EveMinutes
350
300
250
200
150
100
50
0

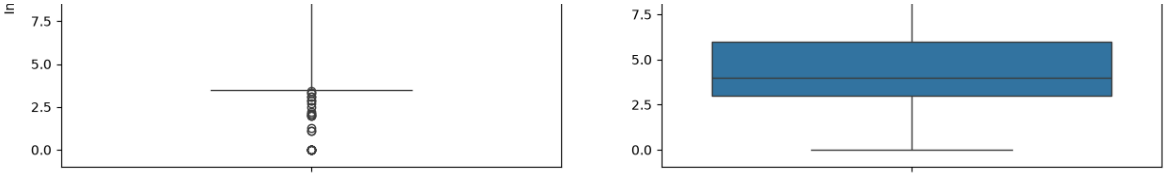

Distribution of 'EveCalls'
EveCalls
175
150
125
100
75
50
25
0


Distribution of 'NightMinutes'
NightMinutes
400
350
300
250
200
150
100
50
0


Distribution of 'NightCalls'
NightCalls
175
150
125
100
75
50
25
0


Distribution of 'IntlMinutes'
20.0
17.5
15.0
12.5
10.0


Distribution of 'IntlCalls'
IntlCalls
17.5
15.0
12.5
10.0



- Some values fall outside the typical range, but none of them look like data errors.
- Most features are roughly symmetric, except `NumVoiceMailMessages`. Let's plot it only for customers with a voicemail plan.

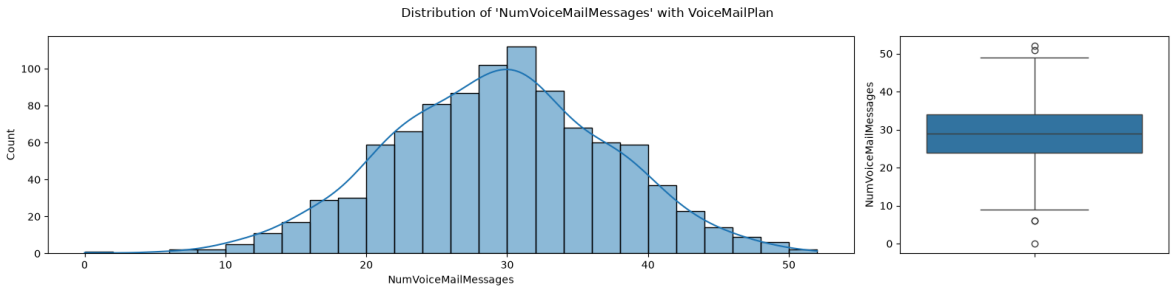
In [14]:
```
voice_messages_with_plan = df[df["VoiceMailPlan"] == 1.0]["NumVoiceMailMe
voice_messages_with_plan

fig, axes = plt.subplots(
    1, 2,
    figsize=(16, 4),
    gridspec_kw={'width_ratios': [3, 1]} # ratio of graphs
    )

sns.histplot(
    data=voice_messages_with_plan,
    ax=axes[0],
    kde=True  # add a smooth line
)

sns.boxplot(
    data=voice_messages_with_plan,
    ax=axes[1])

fig.suptitle(f"Distribution of 'NumVoiceMailMessages' with VoiceMailPlan"
fig.tight_layout()
```



- Now this distribution looks symmetric as well.

4. Data cleaning

- There are no missing values, duplicate rows, or obvious outliers.

5. Preprocessing

5.a Varible encoding

- We already encoded binary features as 0/1. We defined the `encode_binary_features` function for this purpose.

5.b Feature selection

- We excluded the per-period charges because they mirror the per-period minutes.
- We excluded `AreaCode` because it showed no noticeable impact on churn.
- We excluded `State` because it has too many categories (51), and its apparent impact on churn probably just reflects other characteristics of the customers in those states.

In [15]:
```
# initial features to use
raw = [
    "AccountLength", "InternationalPlan", "VoiceMailPlan", "NumVoiceMailM
    "DayMinutes", "DayCalls", "EveMinutes", "EveCalls", "NightMinutes",
    "NightCalls", "IntlMinutes", "IntlCalls", "CustomerServiceCalls"
]
```

5.c Feature engineering

- Let's define a function that creates new features based on our findings, along with some simple aggregates and ratios (e.g., total minutes and the share of minutes used during the day).

In [16]:
```
def add_features(df):

    period_minutes = df["DayMinutes"] + df["EveMinutes"] + df["NightMinut

    total_minutes = period_minutes + df["IntlMinutes"]

    total_calls = (
        df["DayCalls"] + df["EveCalls"] + df["NightCalls"] + df["IntlCall
    )

    total_charge = (
        df["DayCharge"] + df["EveCharge"] + df["NightCharge"] + df["IntlC
    )

    return df.assign(
        # totals
        TotalCharge=total_charge,
        TotalMinutes=total_minutes,
        TotalCalls=total_calls,

        # service call thresholds
        ServiceCalls4Plus=(df["CustomerServiceCalls"] >= 4).astype(float)
        ServiceCalls8Plus=(df["CustomerServiceCalls"] >= 8).astype(float)

        # international usage of customers with plan
        IntlCallsWithPlan=df["IntlCalls"] * df.InternationalPlan,
```

```
        IntlMinutesWithPlan=df["IntlMinutes"] * df.InternationalPlan,

        # share of minutes by period
        DayMinutesShare=df["DayMinutes"].div(period_minutes).fillna(0.0),
        NightMinutesShare=df["NightMinutes"].div(period_minutes).fillna(0
    )


extended_df = add_features(df)
new_columns = [name for name in extended_df.columns if name not in df.col

# all features
columns = raw + new_columns
```

- Print statistics of all training features.

In [17]:
```
X, y = extended_df[columns], extended_df.Churn
X.describe().T[['mean', 'std', 'min', 'max']].round(2)
```

Out[17]:

	mean	std	min	max
AccountLength	100.00	39.47	1.00	243.00
InternationalPlan	0.09	0.29	0.00	1.00
VoiceMailPlan	0.28	0.45	0.00	1.00
NumVoiceMailMessages	8.11	13.73	0.00	52.00
DayMinutes	180.29	53.76	0.00	351.50
DayCalls	100.02	19.85	0.00	163.00
EveMinutes	200.76	50.56	0.00	361.80
EveCalls	100.18	19.80	0.00	170.00
NightMinutes	199.98	50.13	0.00	395.00
NightCalls	99.67	20.06	0.00	175.00
IntlMinutes	10.30	2.72	0.00	20.00
IntlCalls	4.42	2.46	0.00	19.00
CustomerServiceCalls	1.57	1.30	0.00	9.00
TotalCharge	59.50	10.28	25.52	93.39
TotalMinutes	591.34	88.17	284.30	882.20
TotalCalls	304.28	34.50	191.00	416.00
ServiceCalls4Plus	0.08	0.27	0.00	1.00
ServiceCalls8Plus	0.00	0.03	0.00	1.00
IntlCallsWithPlan	0.43	1.52	0.00	15.00
IntlMinutesWithPlan	1.01	3.21	0.00	20.00
DayMinutesShare	0.31	0.08	0.00	0.59
NightMinutesShare	0.34	0.07	0.00	0.67

5.d Data scalling

Since our features are roughly symmetric, it makes sense to use `StandardScaler` in our pipelines.

6. Pipelines

6.1 Pipeline 1 - PCA + XGBoost

Here we try different XGBoost hyperparameters.

In [18]:
```
# IDentify numerical features
nunique = X.nunique()
num_cols = nunique[nunique > 2].index.tolist()
```

```python
# Create numerical pipeline
num_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA())
])

# Standardize numerical features
preprocessor = ColumnTransformer(
    [("num", num_pipe, num_cols)],
    remainder="passthrough",
)

xgb_pipeline = Pipeline([
    ("prep", preprocessor),
    ("xgbclassifier", XGBClassifier()),
])

# hyperparameter search cv
outer_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

# Randomized search
range_eval_metric = ['logloss', 'auc']
range_n_estimators = [10, 50, 100, 150, 200, 300, 500]
range_learning_rate = [0.001, 0.005, 0.01, 0.05, 0.1, 0.3, 0.5, 0.7, 0.9]
range_max_depth = [None, 1, 2, 3, 4, 5, 6, 7, 8]
range_reg_lambda = np.arange(0, 3.1, 0.2)
range_reg_alpha = np.arange(0, 2.1, 0.2)
range_gamma = np.arange(0.1, 4, 0.2)
range_min_child_weight = np.arange(0, 22, 2)
range_subsample = np.arange(0, 1.1, 0.1)
range_colsample_bytree = np.arange(0.1, 1.1, 0.1)
range_n_components = np.arange(5, 31, 5)

param_grid = {
    "prep__num__pca__n_components": np.arange(0.1, 0.99, 0.05),
    'xgbclassifier__eval_metric': range_eval_metric,
    'xgbclassifier__n_estimators': range_n_estimators,
    'xgbclassifier__learning_rate': range_learning_rate,
    'xgbclassifier__max_depth': range_max_depth,
    'xgbclassifier__reg_lambda': range_reg_lambda,
    'xgbclassifier__reg_alpha': range_reg_alpha,
    'xgbclassifier__gamma': range_gamma,
    'xgbclassifier__min_child_weight': range_min_child_weight,
    'xgbclassifier__subsample': range_subsample,
    'xgbclassifier__colsample_bytree': range_colsample_bytree
}

xgb_search = RandomizedSearchCV(
    xgb_pipeline,
    n_iter=100,
    param_distributions=param_grid,
    cv=outer_cv,
    scoring='f1_macro',
    random_state=42
)

xgb_search = xgb_search.fit(X, y)

print(xgb_search.best_params_)
```

```
best_xgb = xgb_search.best_estimator_

print(f'CV F1 macro: {xgb_search.best_score_:.3f}')
```

```
{'xgbclassifier__subsample': np.float64(1.0), 'xgbclassifier__reg_lambda':
np.float64(2.6), 'xgbclassifier__reg_alpha': np.float64(0.8), 'xgbclassifi
er__n_estimators': 300, 'xgbclassifier__min_child_weight': np.int64(8), 'x
gbclassifier__max_depth': None, 'xgbclassifier__learning_rate': 0.1, 'xgbc
lassifier__gamma': np.float64(0.30000000000000004), 'xgbclassifier__eval_m
etric': 'auc', 'xgbclassifier__colsample_bytree': np.float64(0.5), 'prep__
num__pca__n_components': np.float64(0.8000000000000003)}
CV F1 macro: 0.868
```

Here we experiment only with PCA and use default hyperparameters for XGBoost

In [19]:
```
# IDentify numerical features
nunique = X.nunique()
num_cols = nunique[nunique > 2].index.tolist()

# Create numerical pipeline
num_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA())
])

# Standardize numerical features
preprocessor = ColumnTransformer(
    [("num", num_pipe, num_cols)],
    remainder="passthrough",
)

xgb_pipeline = Pipeline([
    ("prep", preprocessor),
    ("xgbclassifier", XGBClassifier()),
])

# hyperparameter search cv
outer_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

param_grid = {
    "prep__num__pca__n_components": np.arange(0.1, 0.99, 0.05)
}

xgb_search = GridSearchCV(
    xgb_pipeline,
    param_grid=param_grid,
    cv=outer_cv,
    scoring='f1_macro'
)

xgb_search = xgb_search.fit(X, y)

print(xgb_search.best_params_)

best_xgb = xgb_search.best_estimator_

print(f'CV F1 macro: {xgb_search.best_score_:.3f}')
```

```
{'prep__num__pca__n_components': np.float64(0.9000000000000002)}
CV F1 macro: 0.892
```

We tried finding the best model using Randomized search, but found that default hyperparameters for the XGBoost classifier worked the best.

6.2 Pipeline 2 - SFS + SVM

We compare different SVM kernels, including the Laplacian kernel, which uses the Manhattan (L1) distance.

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.laplacian_kernel.html

In [20]:
```
# hyperparameter search cv
outer_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
# feature selection cv
inner_cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=1)

# standard scaling + sequential feature selection + svm
pipeline = Pipeline([
    ('scale', StandardScaler()),
    ('select', SequentialFeatureSelector(SVC(), n_features_to_select='aut
    ('svm', SVC()),
])

# laplacian kernel candidate (not built-in)
def laplacian(gamma):
    return lambda X, Y: laplacian_kernel(X, Y, gamma=gamma)

# all candidates, format: (kernel, settings, model)
candidates = (
    [ # linear kernel
        ('linear', f'C={C}, class_weight={weight}',
        SVC(kernel='linear', C=C, class_weight=weight))
        for C in [0.1, 1] for weight in [None, 'balanced']
    ]
    + [ # radial basis function kernel
        ('rbf', f'C={C}, gamma={gamma}',
        SVC(kernel='rbf', C=C, gamma=gamma))
        for C in [10, 100] for gamma in [0.05, 0.1, 0.2]
    ]
    + [ # polynomial kernel
        ('poly', f'C={C}, degree={degree}',
        SVC(kernel='poly', C=C, degree=degree, gamma=0.2, coef0=1))
        for C in [1, 10] for degree in [2, 3]
    ]
    + [ # laplacian kernel
        ('laplacian', f'C={C}, gamma={gamma}',
        SVC(kernel=laplacian(gamma), C=C))
        for C in [30, 100, 300] for gamma in [0.02, 0.05, 0.1]
    ]
)

svm_search = GridSearchCV(
    estimator=pipeline,
    param_grid=[
        {'select__estimator': [model], 'svm': [model]}
        for _, _, model in candidates
    ],
```

```
    scoring={
        'f1_macro': 'f1_macro',
        'precision': make_scorer(precision_score, zero_division=0),
        'recall': 'recall'
    },
    refit='f1_macro',  # choose the best params by f1
    cv=outer_cv,
    n_jobs=-1,
).fit(X, y)

final_svm_model = svm_search.best_estimator_

kernel, settings, _ = candidates[svm_search.best_index_]
print(f'Candidates: {len(candidates)}')
print(f'Best: {kernel}, {settings}')
print(f'CV F1 macro: {svm_search.best_score_:.3f}')
```

```
Candidates: 23
Best: laplacian, C=300, gamma=0.02
CV F1 macro: 0.953
```

- Print a dataframe with the results.

In [21]:
```
results = pd.DataFrame({
    'kernel': [kernel for kernel, _, _ in candidates],
    'settings': [settings for _, settings, _ in candidates],
    'F1 macro': svm_search.cv_results_['mean_test_f1_macro'],
    'std across folds': svm_search.cv_results_['std_test_f1_macro'],
    'precision': svm_search.cv_results_['mean_test_precision'],
    'recall': svm_search.cv_results_['mean_test_recall'],
}).sort_values('F1 macro', ascending=False)

results.round(3).reset_index(drop=True)
```

Out[21]:

	kernel	settings	F1 macro	std across folds	precision	recall
0	laplacian	C=300, gamma=0.02	0.953	0.008	0.995	0.853
1	laplacian	C=100, gamma=0.02	0.952	0.008	0.995	0.851
2	laplacian	C=30, gamma=0.1	0.952	0.008	0.995	0.851
3	laplacian	C=30, gamma=0.05	0.952	0.008	0.995	0.851
4	laplacian	C=100, gamma=0.1	0.952	0.006	0.995	0.851
5	laplacian	C=300, gamma=0.1	0.952	0.007	0.993	0.851
6	laplacian	C=100, gamma=0.05	0.952	0.007	0.993	0.851
7	laplacian	C=300, gamma=0.05	0.951	0.008	0.991	0.851
8	laplacian	C=30, gamma=0.02	0.950	0.010	0.990	0.846
9	rbf	C=10, gamma=0.2	0.937	0.009	0.960	0.830
10	rbf	C=100, gamma=0.05	0.936	0.011	0.967	0.824
11	poly	C=10, degree=3	0.934	0.006	0.958	0.824
12	poly	C=1, degree=3	0.933	0.013	0.960	0.820
13	rbf	C=100, gamma=0.1	0.933	0.009	0.947	0.828
14	rbf	C=100, gamma=0.2	0.931	0.009	0.949	0.822
15	rbf	C=10, gamma=0.1	0.929	0.007	0.949	0.814
16	rbf	C=10, gamma=0.05	0.914	0.011	0.955	0.768
17	poly	C=10, degree=2	0.903	0.016	0.932	0.749
18	poly	C=1, degree=2	0.872	0.015	0.901	0.685
19	linear	C=0.1, class_weight=balanced	0.749	0.016	0.463	0.848
20	linear	C=1, class_weight=balanced	0.748	0.014	0.463	0.844
21	linear	C=1, class_weight=None	0.526	0.079	0.182	0.103
22	linear	C=0.1, class_weight=None	0.526	0.079	0.182	0.103

- All 9 models with a Laplacian kernel perform better than the other variations.

- Which columns does the best model use?

In [22]:
```
final_svm_features = list(final_svm_model['select'].get_feature_names_out
final_svm_features
```

Out[22]:
```
['NumVoiceMailMessages',
 'CustomerServiceCalls',
 'TotalCharge',
 'IntlCallsWithPlan',
 'IntlMinutesWithPlan']
```

7. Confusion matrix

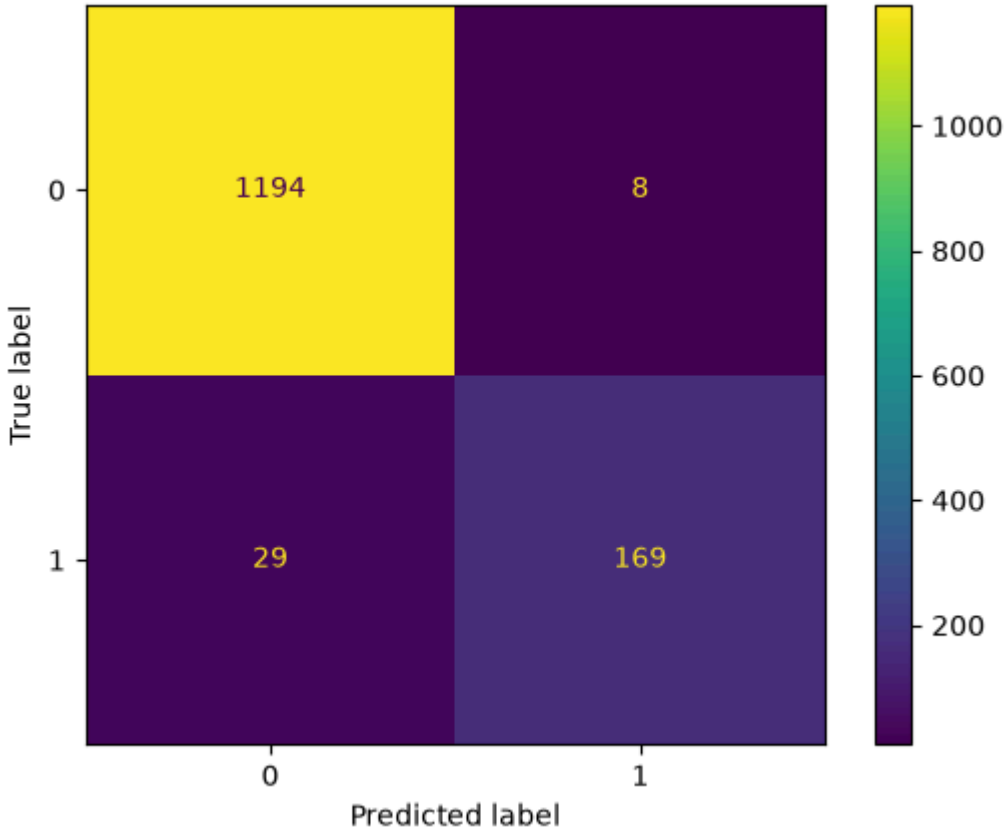
7.1 SFS + SVM

In [23]:
```
X_svm = X[final_svm_features]
X_train, X_test, y_train, y_test = train_test_split(X_svm, y, test_size=0

best_svm = SVC(kernel=laplacian(0.02), C=300)

best_svm.fit(X_train, y_train)
y_pred = best_svm.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
visual = ConfusionMatrixDisplay(confusion_matrix=cm)
visual.plot()
plt.show()
```



29 false negative and 8 false positive

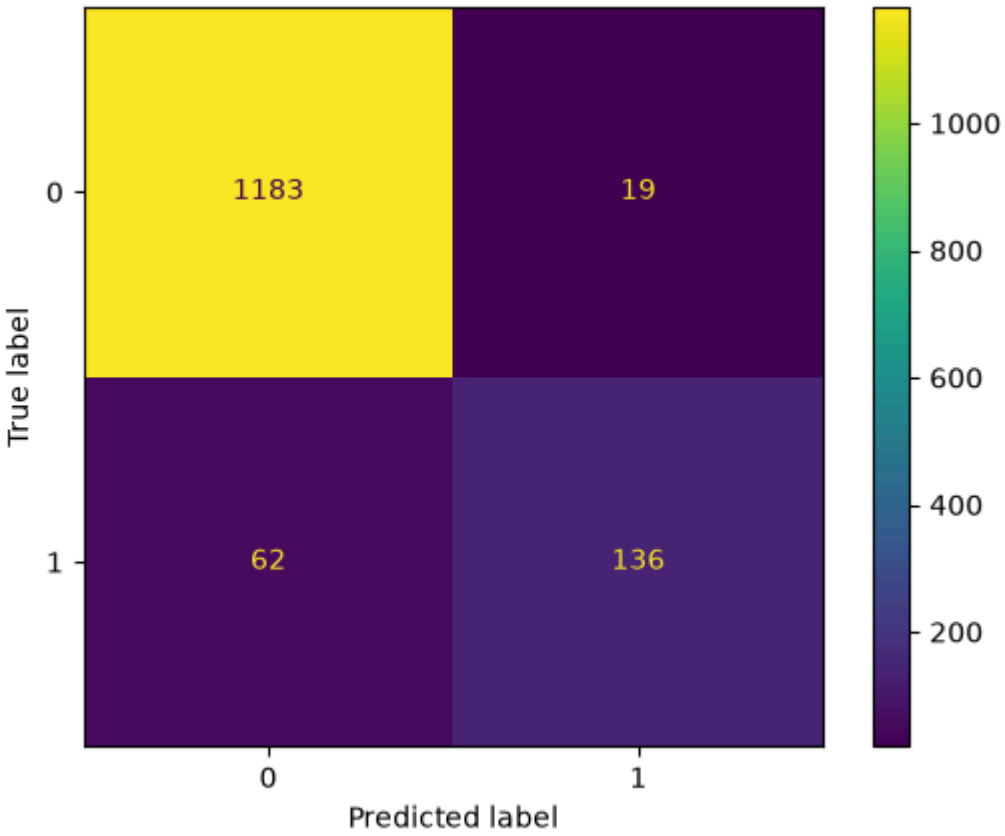
7.2 PCA + XGBoost

In [24]:
```
X_xgb = X
X_train, X_test, y_train, y_test = train_test_split(X_xgb, y, test_size=0

best_xgb.fit(X_train, y_train)
y_pred = best_xgb.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
visual = ConfusionMatrixDisplay(confusion_matrix=cm)
```

```
visual.plot()
plt.show()
```



62 false negative and 19 false positive

8. Kaggle submission

8.1 Test data reading

In [25]:
```
df_test = pd.read_csv("data/testCA4DAT200.csv", index_col="CustomerId")
df_test = encode_binary_features(df_test)
df_test = add_features(df_test)
```

8.2 XGBoost

In [26]:
```
best_xgb.fit(X_xgb, y)

X_test = df_test[columns]
y_pred = best_xgb.predict(X_test)

submission = pd.DataFrame(y_pred, index=X_test.index, columns=["Churn"])
submission.to_csv("data/xgboost_submission.csv", index=True)
```

8.3 SVM

In [27]:
```
X_test = df_test[final_svm_features]

best_svm = SVC(kernel=laplacian(0.02), C=300)
```

```
best_svm.fit(X_svm, y)

y_pred = best_svm.predict(X_test)

submission = pd.DataFrame(y_pred, index=X_test.index, columns=["Churn"])
submission.to_csv("data/svm_submission.csv", index=True)
```